

**Laxmi Charitable Trust's**  
**Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce**  
**Department of Information Technology (B.Sc.I.T Semester V)**  
**Dot Net Core and Progressive Web Application Practical**

## Practical – II

|                              |                                   |
|------------------------------|-----------------------------------|
| Roll No.: T032               | Name: KHUSHI                      |
| Class: TYIT                  | Batch: 2                          |
| Date of Assignment: 09-07-26 | Date/Time of Submission: 09-07-26 |

### PART – A

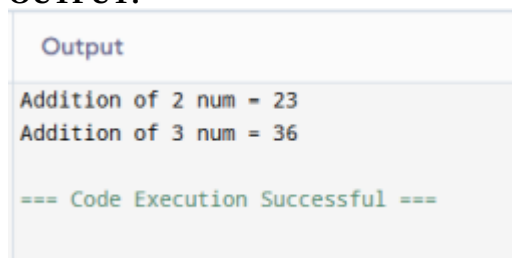
Write the program with different features of C#

#### A] FUNCTION OVER LOADING

**CODE:-**

```
using System;
public class abc
{
    void fun1(int num , int num1)
    {
        Console.WriteLine("Addition of 2 num = " +(num+num1));
    }
    void fun1(int num , int num1 , int num2)
    {
        Console.WriteLine("Addition of 3 num = " +(num+num1+num2));
    }
    public static void Main(string[] args)
    {
        abc obj = new abc();
        obj.fun1(11,12);
        obj.fun1(11,12,13);
    }
}
```

**OUTPUT:-**



The screenshot shows a code execution window with the following output:

```
Output
Addition of 2 num = 23
Addition of 3 num = 36

=== Code Execution Successful ===
```

#### B] INHERITANCE (all types )

##### SINGLE INHERITANCE

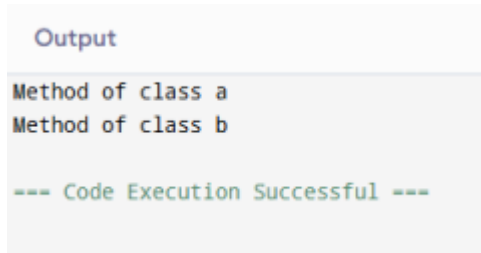
**CODE:-**

```
using System;
public class a {
    public
    void fun1() {
        Console.WriteLine("Method of class a "); } }
```

```

public class b : a {
    public
    void fun2() {
        Console.WriteLine("Method of class b
"); } }
public class main1 {
    public static void Main(string[] args) {
        b obj = new b();
        obj.fun1();
        obj.fun2(); } }

```

**OUTPUT:-**


The screenshot shows a window titled "Output" with the following text:

```

Method of class a
Method of class b

--- Code Execution Successful ---

```

**MULTILEVEL INHERITANCE****CODE:-**

```
using System;
```

```

public class A
{
    public void fun1()
    {
        Console.WriteLine("Class A is here!");
    }
}

```

```

public class B : A
{
    public void fun2()
    {
        Console.WriteLine("Class B is here!");
    }
}

```

```

public class C : B
{
    public void fun3()
    {
        Console.WriteLine("Class C is here!");
    }
}

```

```

public class Program
{
    public static void Main(string[] args)
    {
        C obj = new C();
    }
}

```

```

        obj.fun1();
        obj.fun2();
        obj.fun3();
    }
}

```

**OUTPUT:-**

```

Class A is here!
Class B is here!
Class C is here!

=== Code Execution Successful ===

```

**HIERARCHICAL INHERITANCE****CODE:-**

```
using System;
```

```

public class A
{
    public void fun1()
    {
        Console.WriteLine("Class A is here!");
    }
}

```

```

public class B : A
{
    public void fun2()
    {
        Console.WriteLine("Class B is here!");
    }
}

```

```

public class C : A
{
    public void fun3()
    {
        Console.WriteLine("Class C is here!");
    }
}

```

```

public class Program
{
    public static void Main(string[] args)
    {
        B obj = new B();
        obj.fun1();
        obj.fun2();

        C obj1 = new C();
        obj1.fun1();
        obj1.fun3(); // Correct
    }
}

```

**OUTPUT:-**

```

Class A is here!
Class B is here!
Class A is here!
Class C is here!

=== Code Execution Successful ===

```

**MULTIPLE INHERITANCE****CODE:-**

```

using System;
public interface A
{
    public void fun1();
}
public interface B
{
    public void fun2();
}
public class C: A,B
{
    public void fun1()
    {
        Console.WriteLine("Interface A is here!");
    }
    public void fun2()
    {
        Console.WriteLine("Interface B is here!");
    }
    public void fun3()
    {
        Console.WriteLine("Class C is here!");
    }
}
public class Program
{
    public static void Main(string[] args)
    {
        C obj1 = new C();
        obj1.fun1();
        obj1.fun2();
        obj1.fun3();
    } }

```

**OUTPUT:-**

```

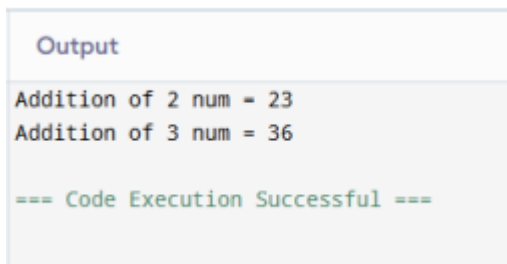
Interface A is here!
Interface B is here!
Class C is here!

=== Code Execution Successful ===

```

**C] CONSTRUCTOR OVERLOADING****CODE:-**

```
using System;
public class abc {
    abc(int num , int num1) {
        Console.WriteLine("Addition of 2 num = "
            +(num+num1)); }
    abc(int num , int num1 , int num2) {
        Console.WriteLine("Addition of 3 num = "
            +(num+num1+num2)); }
    public static void Main(string[] args) {
        abc obj = new abc(11,12);
        abc obj1 = new abc(11,12,13); } }
```

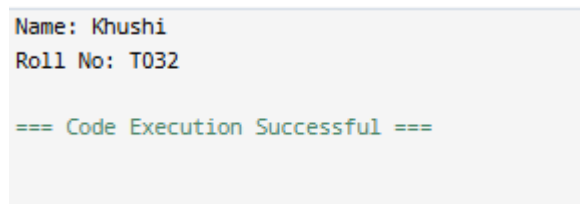
**OUTPUT:-**


The screenshot shows the output of the C# program. It displays two lines of text: "Addition of 2 num = 23" and "Addition of 3 num = 36". Below these, it says "=== Code Execution Successful ===".

**D] INTERFACES****CODE:-**

```
using System;

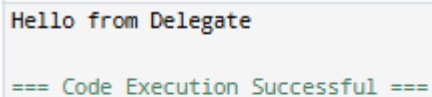
interface IStudent
{
    void display();
}
class Student : IStudent
{
    public void display()
    {
        Console.WriteLine("Name: Khushi");
        Console.WriteLine("Roll No: T032");
    }
}
class Program
{
    public static void Main(string[] args)
    {
        Student obj = new Student();
        obj.display();
    } }
```

**OUTPUT:-**


The screenshot shows the output of the C# program. It displays two lines of text: "Name: Khushi" and "Roll No: T032". Below these, it says "=== Code Execution Successful ===".

**E] USING DELEGATES & EVENTS****CODE:-**

```
using System;
public delegate void Demo();
class Program
{
    public static void Show()
    {
        Console.WriteLine("Hello from Delegate");
    }
    static void Main(string[] args)
    {
        Demo d = Show;
        d();
    }
}
```

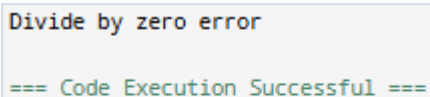
**OUTPUT:-**

```
Hello from Delegate

=== Code Execution Successful ===
```

**F] EXCEPTION HANDLING****CODE:-**

```
using System;
class Program
{
    static void Main(string[] args)
    {
        try
        {
            int a = 10;
            int b = 0;
            Console.WriteLine(a / b);
        }
        catch
        {
            Console.WriteLine("Divide by zero error");
        }
    }
}
```

**OUTPUT:-**

```
Divide by zero error

=== Code Execution Successful ===
```

